

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

**Отчёт
Домашнее задание № 3**

Выполнил:
Ермаков Максим Олегович
K3340

Проверил:
Добряков Д. И.

Санкт-Петербург
2025 г.

Задача

Реализовать тестирование разработанного REST API средствами Postman и подготовить единый комплексный сценарий, покрывающий основной бизнес-процесс приложения для бронирования столиков в ресторанах.

В рамках выполнения задания необходимо было:

- реализовать тестирование API средствами Postman;
- написать автоматические тесты внутри Postman для каждого шага сценария;
- получить один комплексный сценарий по основному процессу приложения, воспроизводимый при многократном запуске.

Ход работы

Описание тестируемого backend-приложения

Тестируемое приложение реализует REST API платформы бронирования ресторанов. Оно поддерживает широкий спектр функциональности, включая работу как с обычными пользователями, так и с администраторами. Среди реализованных функций:

- регистрация и авторизация пользователей;
- получение профиля пользователя;
- поиск ресторанов с фильтрацией;
- просмотр детальной карточки ресторана, меню, фотографий и отзывов;
- проверка доступности столиков и создание бронирования;
- просмотр истории бронирований и создание отзывов;
- административное создание и публикация ресторанов, управление локациями, столиками, меню, фотографиями и статусами бронирований.

Использованные технологии и инструменты

В ходе работы использовались следующие технологии и инструменты:

- Postman — инструмент для ручного и автоматизированного тестирования API;
- REST API — архитектурный стиль взаимодействия клиента с сервером;
- JSON — формат передачи данных в запросах и ответах;
- JWT Bearer Authentication — механизм авторизации на основе токенов.

Разработка комплексного тестового сценария

На первом этапе было выбрано основное бизнес-направление тестирования — основной пользовательский процесс приложения бронирования ресторанов. Был сформирован комплексный сценарий, охватывающий весь путь от регистрации пользователя до завершения и подтверждения бронирования.

Реализованный сценарий состоит из следующих последовательных шагов:

1. Регистрация нового пользователя.
2. Вход пользователя в систему и получение JWT-токена.
3. Получение профиля текущего пользователя.
4. Вход администратора и получение административного токена.
5. Получение списка кухонь.
6. Создание локации администратором.
7. Создание ресторана администратором.
8. Добавление столика к ресторану.
9. Создание категории меню.
10. Добавление блюда в категорию меню.
11. Добавление фотографии ресторана.
12. Публикация ресторана администратором.
13. Поиск ресторана по фильтрам.
14. Получение детальной карточки ресторана.
15. Проверка доступности столиков на выбранную дату и время.
16. Создание бронирования пользователем.
17. Получение списка бронирований пользователя.
18. Создание отзыва о ресторане.
19. Получение списка отзывов ресторана.
20. Подтверждение бронирования администратором.
21. Получение бронирования по идентификатору для проверки итогового статуса.

Организация автоматизации в Postman

В Postman была создана коллекция запросов, для которой настроено локальное окружение с базовым URL и учётными данными администратора. Переменная `baseUrl` вынесена на уровень окружения, что позволяет быстро переключаться между средами без изменения самих запросов.

Для обеспечения связности сценария были активно использованы `collection variables` и `environment variables`. После выполнения каждого запроса авторизации токены пользователя и администратора автоматически сохранялись через `pm.collectionVariables.set(...)`. Аналогичным образом сохранялись идентификаторы всех созданных сущностей: локации, ресторана, столика, категории меню, блюда, фотографии, бронирования и отзыва. Это позволило обеспечить передачу данных между шагами без ручного копирования значений.

Реализованные автоматические проверки

Для каждого шага сценария в разделе Tests были добавлены автоматические проверки. В совокупности они охватывают следующие аспекты корректности работы API:

- проверка HTTP-статуса ответа (200, 201 и т.д.) для каждого запроса;
- проверка наличия обязательных полей в JSON-ответе сервера;
- проверка роли пользователя после авторизации;
- проверка того, что созданные сущности действительно возвращаются сервером;

- проверка присутствия созданного ресторана в результатах поиска по фильтрам;
- проверка того, что столик доступен до создания бронирования;
- проверка того, что бронирование создаётся со статусом PENDING;
- проверка присутствия созданного бронирования в истории пользователя;
- проверка успешного добавления отзыва;
- проверка того, что администратор изменяет статус бронирования на CONFIRMED;
- проверка того, что пользователь видит подтверждённый статус своего бронирования.

Обеспечение повторного запуска коллекции

В ходе работы была выявлена проблема: при повторном прогоне коллекции часть тестов завершалась с ошибкой, поскольку в базе данных уже существовали сущности с одинаковыми данными (дублирующиеся email, названия ресторанов, локации и т.д.).

Для решения этой проблемы был реализован механизм генерации уникального суффикса на основе текущего времени. В Pre-request Script первого запроса коллекции формировалось значение suffix, которое затем использовалось для создания уникальных рабочих переменных через `pm.collectionVariables.set(...)`. К уникальным переменным были отнесены:

- `locationCityRun` — город локации;
- `locationAddressRun` — адрес локации;
- `restaurantTitleRun` — название ресторана;
- `tableNumberRun` — номер столика;
- `menuCategoryTitleRun` — название категории меню;
- `menuItemTitleRun` — название блюда.

Также к уникальным данным был отнесён email и телефон регистрируемого пользователя. Данный подход позволил сделать тестовый сценарий полностью воспроизводимым и независимым от результатов предыдущих запусков без необходимости ручной очистки базы данных.

Состав подготовленных файлов

По итогам работы были подготовлены следующие файлы:

- коллекция Postman с основным сценарием тестирования;
- `environment` с локальными параметрами подключения (`baseUrl`, учётные данные администратора);
- README с краткой инструкцией по запуску коллекции.

Результат

В результате выполнения домашнего задания была создана коллекция Postman, реализующая комплексный автоматизированный сценарий тестирования основного

бизнес-процесса приложения для бронирования ресторанов. Коллекция включает автоматические проверки корректности ответов API, использование переменных окружения и коллекции для передачи токенов и идентификаторов между запросами, а также механизм генерации уникальных тестовых данных для обеспечения повторного запуска.

Подготовленный сценарий позволяет последовательно проверить ключевые функции системы: регистрацию и авторизацию пользователей, создание ресторанной инфраструктуры администратором, поиск и просмотр ресторанов, создание и подтверждение бронирования, а также работу с отзывами. Таким образом, в одном прогоне коллекции охватывается весь основной пользовательский путь приложения.

Вывод

Цель домашнего задания достигнута. В ходе работы было освоено тестирование REST API средствами Postman: от организации окружения и переменных коллекции до написания автоматических тестов и обеспечения воспроизводимости сценария.

Ключевым результатом стала не просто последовательность ручных запросов, а полноценный автоматизированный тестовый сценарий, покрывающий весь основной бизнес-процесс приложения — от регистрации пользователя до подтверждения бронирования. Реализованные тесты повышают удобство проверки API и существенно упрощают демонстрацию работоспособности системы.

Использование механизма уникальных суффиксов позволяет запускать коллекцию многократно без конфликтов с уже существующими данными, что делает её пригодной для использования в качестве регрессионного теста при дальнейшей разработке приложения.